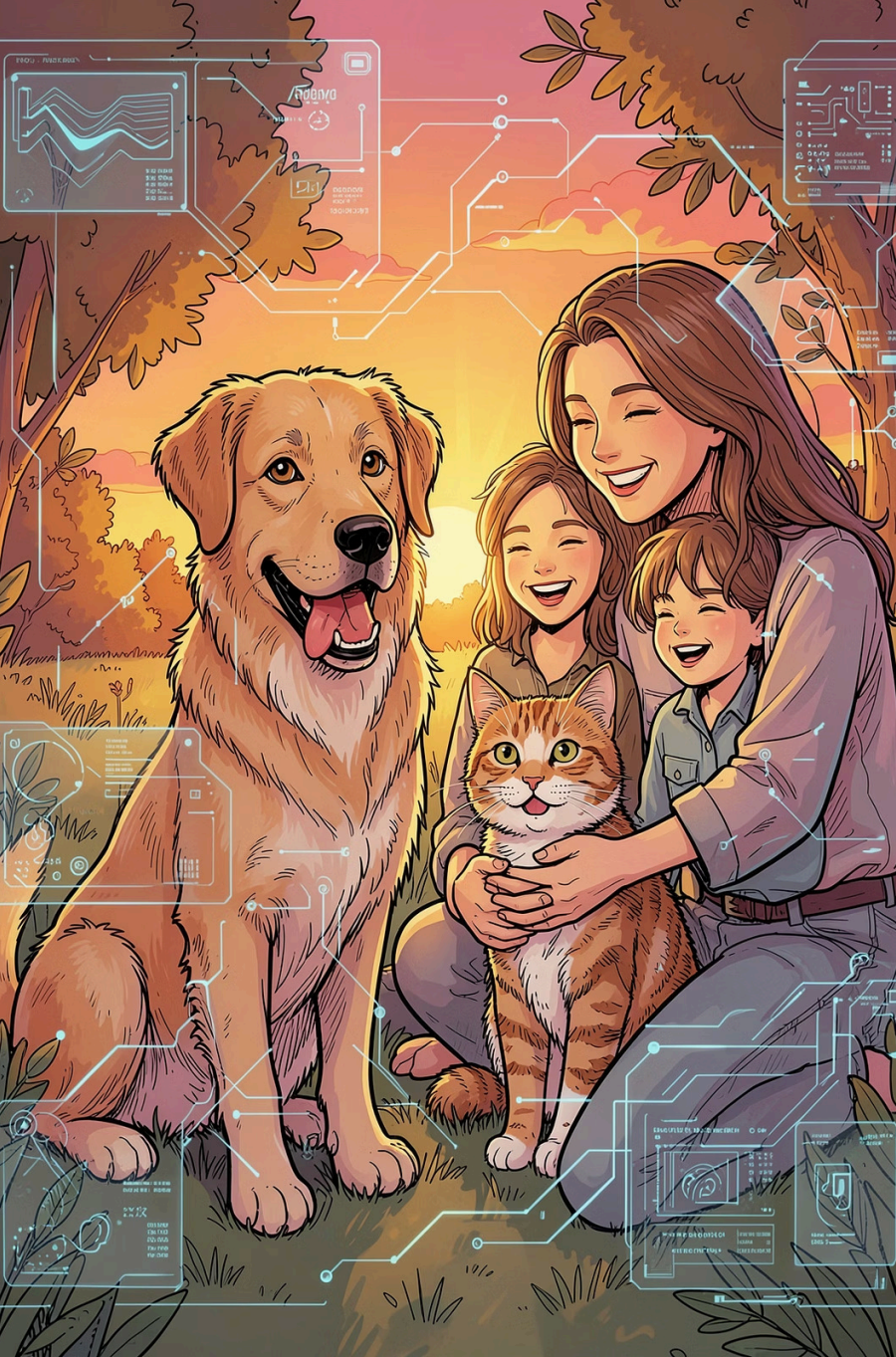




Sanos y Salvos

Plataforma

Inteligente para la

localización y

recuperación de

mascotas perdidas

Presentación de Arquitectura de Microservicios · Bryan Jimenez · Prof.
Viviana Soledad · 08-04-2026

Contexto del Proyecto

El Problema

Creciente pérdida de mascotas y saturación de reportes desorganizados en redes sociales y refugios, generando angustia en los dueños.

La Solución

Plataforma centralizada para dueños, clínicas veterinarias y refugios que automatiza la detección de coincidencias entre mascotas perdidas y encontradas.

El Enfoque

Arquitectura escalable basada en **microservicios**, con tres módulos clave: Gestión de Mascotas, Geolocalización y Motor de Coincidencias.



Arquitectura Global de Microservicios

Ecosistema basado en **Spring Boot** (Backend) y **React** (Frontend), con **API Gateway** como orquestador central y punto único de entrada.

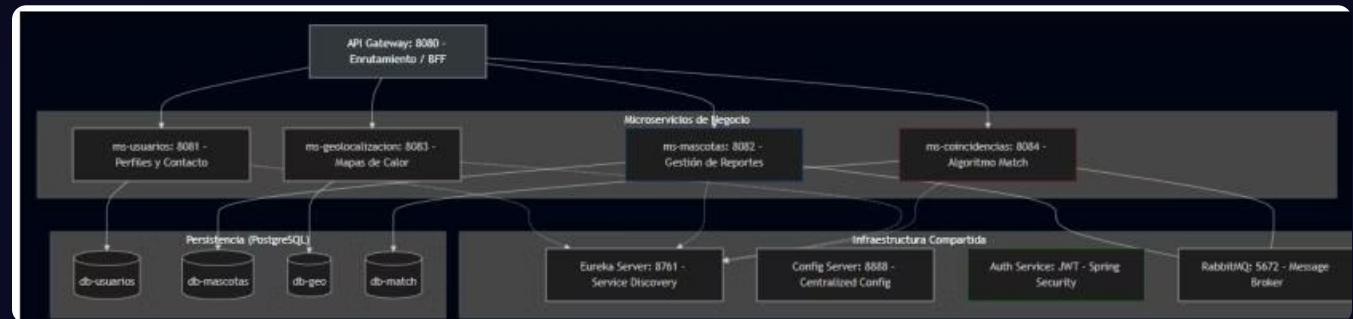
Patrón de **base de datos por servicio** utilizando PostgreSQL containerizado en Docker, garantizando aislamiento e integridad de datos.

SPRING BOOT

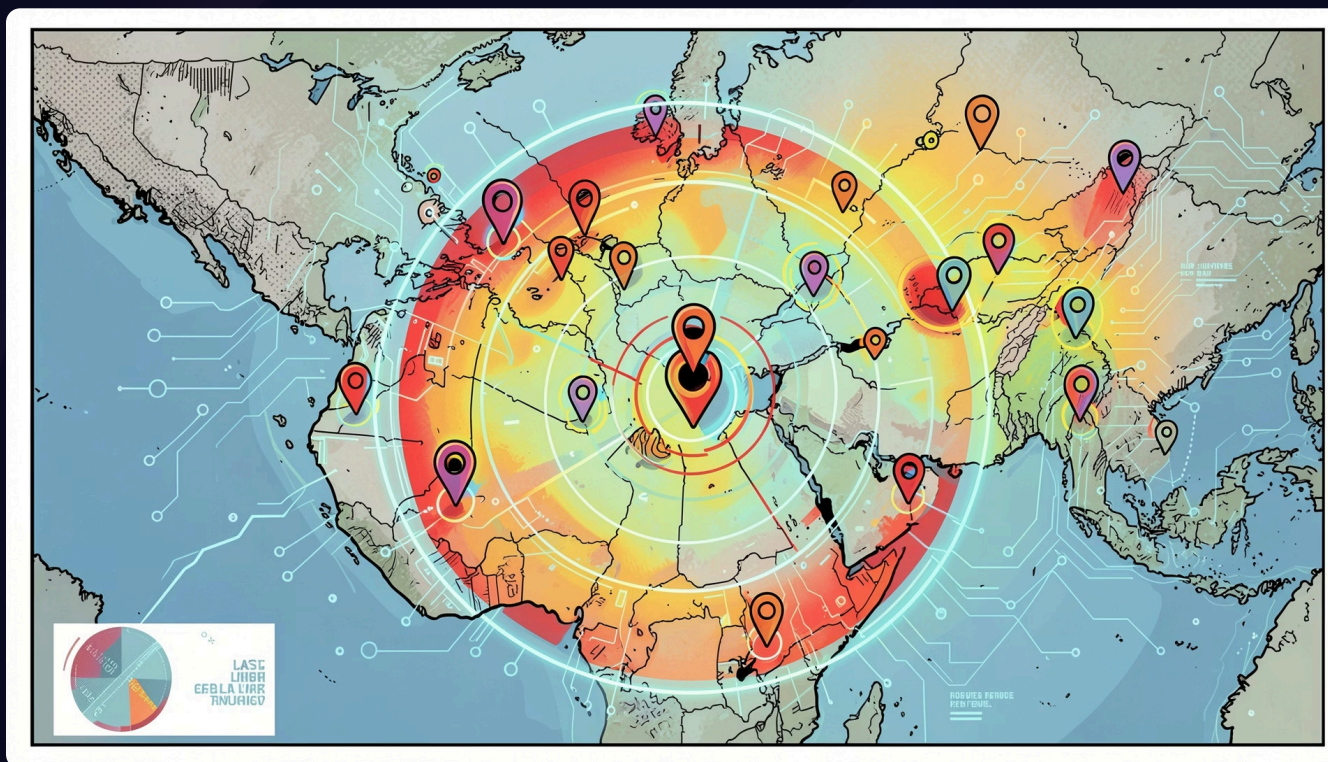
REACT

DOCKER

POSTGRESQL



Motor Espacial y Mensajería Asíncrona



Fórmula de Haversine

Cálculo de distancia geográfica considerando la curvatura terrestre. Gatilla un *match* automático cuando la brecha es ≤ 10 km.

Comunicación Síncrona (OpenFeign)

Peticiones en tiempo real entre microservicios mediante cliente declarativo REST para recuperar información geográfica.

Mensajería Asíncrona (RabbitMQ)

El *ms-reportes* emite eventos a RabbitMQ; el motor de coincidencias los procesa de forma desacoplada sin bloquear al usuario.

Calidad del Software y Seguridad



→ SonarQube — Quality Gates: Passed

Cobertura de pruebas: **ms-usuarios 81,3%**, ms-reportes 73,7%, ms-mascotas 70,0%, ms-coincidencias 68,9%. Cero brechas de seguridad críticas.

→ Circuit Breaker (Resilience4j)

Estados CLOSED, OPEN y HALF-OPEN en el API Gateway con fallback HTTP 503 para prevenir fallos en cascada.

→ Seguridad Perimetral (JWT)

Autenticación centralizada mediante tokens JWT y Spring Security en el API Gateway, único punto de entrada al sistema.

Conclusión y Aprendizaje

Arquitectura Distribuida

Éxito en la implementación de una arquitectura desacoplada con microservicios independientes, cada uno con su propia base de datos PostgreSQL.

Desafíos Superados

Volúmenes persistentes en Docker, comunicación asíncrona con RabbitMQ y configuración de Eureka y API Gateway para orquestación del sistema.

Estándares de Industria

El sistema es **escalable, resiliente y seguro**, cumpliendo con los requerimientos funcionales y técnicos del cliente Sanos y Salvos.

